

Events (1)

Brainster Web Development Academy



Table of contents

01

Event basics

02

addEventListener vs. on*

03

removeEventListener

04

Mouse events

05

Keyboard events



Event basics

- **At the most basic level, a (DOM) event is a way of notifying code that something of interest has happened.**
- **An event can be many things - a click, a button press, an automated notification, the end of a loading process etc.**
- **Similar as with strings, numbers, arrays, and Event is also its own type, with its own properties and methods.**



addEventListener vs. on

- The most commonly used, and most standards-compliant way to attach events, is the use of the **.addEventListener()** method. Each DOM element supports it (plus a number of built-in browser types)
- There is however, another (and older) way of attaching events, colloquially called **on**. The biggest difference between the two is that using the **.on*** method, we can attach only one handler while with **.addEventListener**, we can attach as many as we want.



addEventListener vs. on*

```
function myHandler() {  
  console.log('clicked');  
}
```

```
let btn = document.querySelector('button');  
btn.addEventListener('click', myHandler);  
// The “right” way, for anyone wondering :)
```

```
function myHandler() {  
  console.log('clicked');  
}
```

```
let btn = document.querySelector('button');  
btn.onclick = myHandler;  
// The older, less flexible way
```



Mouse events



The “click” event

- Every DOM element can be made “clickable” (even divs, spans...)
- That said - just because you can, doesn’t mean you should :)
- Most commonly, we attach “click” event listeners to buttons and a elements - since those are the ones that should intuitively be considered interactive.



The “click” event

How to make something clickable:

```
function onClick() {  
  console.log('clicked');  
}
```

7

```
let myButton = document.querySelector('button');  
myButton.addEventListener('click', onClick);
```



Exercise I

1. Create a simple html page with a single div (with a class of “form”) and a single button within that div.
2. Create a JavaScript file called “form.js”, save it and add it inside the html page using a `<script>` tag.
3. Create a function called “onSubmit” which simply alerts “Thank you!” when clicked.
4. Add a click event listener to the button and invoke the onSubmit function when the button gets clicked.



“mouseover” and “mouseout”

```
let myDiv = document.querySelector('div');
```

```
function onMouseOver() {  
    myDiv.style.backgroundColor = 'blue';  
}
```

```
function onMouseOut() {  
    myDiv.style.backgroundColor = 'white';  
}
```

```
myDiv.addEventListener('mouseover', onMouseOver);  
myDiv.addEventListener('mouseout', onMouseOut);
```



Exercise II

1. Using the file and code from Exercise I, add:
 - a) A function called **onMouseOver** that gives the button a css class of “hover”;
 - b) A function called **onMouseOut** that removes the “hover” class from the button;
2. To the button, add an event listener for “mouseover” and invoke onMouseOver.
3. Also add an event listener for “mouseout” and invoke onMouseOut.



“mouseover” and “mouseout” - related events

Similar to click, there are finer-grained events related to “mouseover” and “mouseout” too:

- “**mouseenter**” - element entered (ignoring children)
- “**mouseleave**” - element left (ignoring children)
- “**mousemove**” - pointer movement

12

*The big difference between “**mouseover**” / “**mouseenter**” and “**mouseout**” / “**mouseleave**” is that the former apply to the children elements, while the latter ignore the children elements (if any).*



Keyboard events



“keyup” and “keydown”

- The “keyup” and “keydown” events refer to keyboard buttons being pressed when some element is focused.
- As the name suggests, the former fires when we release a key, while the latter fires while we hold the key down -and it doesn't stop firing while we hold the button :)
- These are the events most commonly used when handling text inputs.



“keyup” and “keydown”

```
let myInput = document.querySelector('input');  
let myValue = '';
```

```
function onKeyUp() {  
  myValue = myInput.value;  
  console.log(myValue);  
}
```

```
mySelect.addEventListener('keyup', onKeyUp);
```



Exercise III

1. Add two more divs with a class of “row” to the document.
2. In the first one, add a label with the text “Name” and an input tag next to the label.
3. In the second one, add a label with the text “About me” and a textarea next to the label.
4. Define two empty variables: name and aboutMe.
5. Define a function called onNameChange which should simply rewrite the name variable with the value from the input field.
6. Define a function called onAboutChange which should rewrite the aboutMe variable with the value from the textarea.
7. Add a “keyup” event listener to both the input and the textarea, triggering onNameChange for the input and onAboutChange for the textarea.



The “keypress” event

- Much like “keyup” and “keydown”, “keypress” also triggers when the keyboard is used. However, unlike the previous two, “keypress” is a bit weird - it triggers both on “keydown”, “keyup” and while we’re holding some key.
- Unless you really understand what you’re doing, use “keyup” and “keydown”, as they tend to be a bit easier to reason about.

